PinkCheck: CLAUDE.md

Hackathon project. Theme: women's everyday challenges, the "Pink Tax". About 1 hour of build time is left, then a strict 2-minute live pitch and 2 minutes of judge feedback. Judging criteria: originality/creativity, live demo delivery, UI/UX, real-world impact, relevance to theme. **Ship a working demo over more features.** If something isn't working by its checkpoint, cut it.

What we're building

PinkCheck is a mobile-first web app that helps shoppers spot when they may be paying more for a product just because it's marketed to women, and shows a cheaper equivalent.

One app, three ways in, one shared comparison engine:

Tab	Owner	What it does
Search	Person 1	Type a product, get a Pink Tax Score, cheaper alternatives and savings
Scan	Person 2	Live rear camera, tap Capture, AI reads the product, same result card titled "Cheaper alternatives" (like a supermarket app's out-of-stock swap suggestions)
Receipt	Person 3	Upload a receipt photo, AI reads the line items, user can fix them, app totals "Pink Tax paid" and "You could have saved"

Person 4 owns `data/demo.json` (real, hand-checked prices) and the pitch.

Non-negotiable rules

1. **Never invent prices, sizes or products.** Prices come only from: (a) `data/demo.json` (hand-checked), (b) the user's own photo (shelf label or receipt), or (c) a Claude web search result that includes a source URL. If none of these exists, show "price not found". Never fill a price from model memory.
2. **The score is computed in code** (`lib/score.ts`), never by the LLM. The LLM only identifies products, finds candidates (with sources) and writes the explanation text.
3. **Only Person 1 edits `lib/score.ts`, `lib/types.ts` and `app/api/compare`.** Scan and Receipt call `/api/compare`; they never compute their own scores.
4. The Anthropic API key stays on the server (`process.env.ANTHROPIC_API_KEY`). Never import the SDK in client components.
5. Hedged, fair wording: "may be paying more", not "is ripping you off". Show reverse gaps honestly ("Good news: the women's version is cheaper here").
6. Every result shows its price source (a link, or "from your photo") and its date checked. Web-search results get a "Live estimate, check source" badge.

Background (for copy and explanations)

- The 2015 NYC Department of Consumer Affairs study found women's personal care products cost about 13% more than men's.

- Moshary, Tuchman & Bhatia (Marketing Science, 2023) found that once you compare products with similar ingredients, there is no systematic premium. Most gaps come from product differences and pack sizes. So PinkCheck's value is a **per-unit comparison of functionally equivalent products**, which shoppers can't easily do in the aisle.
- UK: gender-based pricing of goods isn't illegal. Consumer pressure (#sexistsurcharge) has made Boots and Tesco change prices before. Unit pricing on cosmetics became mandatory under the Price Marking Order reforms from April 2026.
- US: New York (2020) and California (2023, AB 1287) ban gender-based pricing of substantially similar goods.

Tech stack

- Next.js (App Router) + TypeScript + Tailwind. Deploy on Vercel (HTTPS is required for camera access).
- `@anthropic-ai/sdk`. Model: `claude-sonnet-5-5` for everything. Server-side web search tool: `{ type: "web_search_20250305", name: "web_search", max_uses: 3 }` (web search must be enabled for the org in the Claude Console).
- `zod` to validate every LLM JSON response. `fuse.js` for fuzzy matching against `demo.json`.
- No database, no auth.

File structure

```
app/
  layout.tsx                bottom tab bar: Search | Scan | Receipt
  page.tsx                  Search tab (Person 1)
  scan/page.tsx             Scan tab (Person 2)
  receipt/page.tsx          Receipt tab (Person 3)
  api/compare/route.ts      comparison engine (Person 1)
  api/identify/route.ts     image -> Product (Person 2)
  api/receipt/route.ts      receipt image -> line items (Person 3)
components/
  ResultCard.tsx            shared result UI (Person 1 builds it first,
others reuse it)
  ScoreGauge.tsx
  Loading.tsx
lib/
  types.ts                  shared contract (Person 1, frozen after 0:10)
  score.ts                  scoring maths (Person 1 only)
  claude.ts                 Anthropic client + extractJson() helper
  image.ts                  client-side resize to JPEG base64
data/
  demo.json                 hand-checked demo pairs (Person 4)
```

Shared contract (`lib/types.ts`)

typescript

```typescript
export type Unit = 'ml' | 'g' | 'blade' | 'item';
export type Gender = 'female' | 'male' | 'neutral' | 'unknown';

export type Product = {
  name: string;
  brand?: string;
  retailer?: string;
  price_gbp?: number | null;    // null/undefined = unknown, never guessed
  quantity?: number | null;     // e.g. 250 (ml), 8 (blades)
  unit?: Unit;
  gender_target?: Gender;
  source?: 'demo' | 'photo' | 'web' | 'receipt';
  source_url?: string;
  checked_on?: string;          // ISO date
};

export type Comparison = {
  target: Product;
  alternatives: Product[];      // cheapest per unit first, max 3
  best?: Product;               // alternatives[0] if comparable
  equivalence: number;          // 0–1, how functionally equivalent best is
  equivalence_reasons: string[]; // short chips, e.g. "Same 3 blades", "Same 25
  gap: number | null;           // (u_target - u_best) / u_best; negative = reve
  score: number | null;         // 0–100 from lib/score.ts
  label: 'No meaningful gap' | 'Moderate' | 'High' | 'Reverse gap' | 'Not enoug
  savings_per_pack: number | null;
  explanation: string;          // max 60 words, plain English
  live: boolean;                // true when any price came from web search
};

// POST /api/compare  { product: Product }          -> Comparison
// POST /api/identify { image: string /*base64 jpeg*/ } -> { product: Product }
// POST /api/receipt  { image: string }             -> { items: ReceiptItem[];
export type ReceiptItem = Product & { raw_text: string; line_total: number | nu
```

Scoring (`lib/score.ts`)

```
unit_price = price_gbp / quantity            (needs both; else null)
gap        = (u_target − u_best) / u_best
score      = round(100 * min(1, max(0, gap) / 0.5) * equivalence)
savings_per_pack = (u_target − u_best) * target.quantity   (only if gap > 0)
label: gap < 0 -> "Reverse gap"; score < 20 -> "No meaningful gap"; < 50 ->
"Moderate"; else "High"
      missing prices/quantities -> "Not enough data", score null
```

Only compare matching units (ml vs ml, blade vs blade). If units differ, return "Not enough data". Write unit tests (or a quick script) for: a clear gap, equal prices, a reverse gap, missing data.

Feature specs

Search + engine (Person 1)

`/api/compare` flow:

1. Fuzzy-match `product.name` against `data/demo.json` (Fuse.js, threshold ~0.4). If matched and its prices are filled, build the Comparison from the demo pair (`source: 'demo'`, `live: false`) and use its saved `explanation` if present. Instant: the live demo depends on this.
2. Otherwise, call Claude with web search. Prompt: find this exact product at a UK retailer (Boots, Superdrug, Tesco, Sainsbury's, Asda) and up to 3 functionally equivalent products (same function, similar size, same unit type), deliberately including men's, unisex and own-brand versions. Return JSON only with name, brand, retailer, price_gbp, quantity, unit, gender_target and source_url for each, plus equivalence 0-1, equivalence_reasons and an explanation. Rule: never guess a price; omit any product without a source URL.
3. Validate with zod, drop items without `source_url`, compute everything in `lib/score.ts`, and set `live: true`.
4. Timeout of 25 s, then return `label: 'Not enough data'` with a friendly message.
5. Cache results in memory (a `Map` keyed by lowercase name) so repeated demo queries are instant.

`ResultCard` UI: score gauge with label and colour; the target product vs the best alternative side by side (price, size, **unit price in big type**, retailer, source link); `equivalence_reasons` as chips (substitution-style "why this swap" reasons); up to 2 more alternatives in a horizontal swipe row; a savings badge; the explanation; and a "Call it out" button (see extras).

Scan (Person 2)

- Build in this order: (1) `<input type="file" accept="image/*" capture="environment">` fallback (works everywhere, about 10 min); (2) live view: `getUserMedia({ video: { facingMode: 'environment' }, audio: false })` into `<video playsInline muted autoPlay>`, with a big Capture button that draws the frame to a canvas.
- Resize to a max of 1000 px on the long edge, JPEG quality 0.8, base64 (`lib/image.ts`). Claude downsizes images over ~1568 px anyway.
- `/api/identify`: send the image to Claude and ask for JSON `{ name, brand, quantity, unit, gender_target, price_gbp }`. Set `price_gbp` only if a shelf label price is clearly

visible (then `source: 'photo'`); otherwise null. If unreadable, return `{ error }`.

- Show "We think this is: <name> <size>" with an editable text field and a "Find cheaper alternatives" button, then POST to `/api/compare` and render `ResultCard`.
- Do NOT analyse every video frame (too slow and costly). Live viewfinder + Capture is the design.
- Barcode scanning is a stretch only: iOS Safari has no native BarcodeDetector, so it would need the `barcode-detector` polyfill (ZXing WASM). A barcode also gives no price. Skip unless everything else is done.

Receipt (Person 3)

1. Upload a photo (file input, accept image/*), resize to a max of 1500 px, POST to `/api/receipt`.
2. `/api/receipt`: Claude reads every line item and returns JSON. Prompt rules: copy prices exactly as printed, keep `raw_text`, expand abbreviations into a best-guess `name` (e.g. "GIL VNS WMN RZR 3PK" -> "Gillette Venus women's razor 3 pack"), set `unreadable: true` instead of guessing, and classify `gender_target` from the product name.
3. In code: if the line totals don't sum to `receipt_total` within £0.05, show a "Some items may be misread, please check" banner.
4. Show the items in an **editable table** (name, qty, price). Female-targeted personal-care items are pre-ticked.
5. A "Check for Pink Tax" button sends up to 5 ticked items to `/api/compare` in parallel (`Promise.all`) with a per-item progress indicator.
6. Summary card (computed in code): **Pink Tax paid** = sum over items with gap > 0 of `(u_target - u_best) * quantity * qty`; **You could have saved** = the same in £, shown as total; **Yearly estimate** = total × trips per year (editable input, default 12, labelled "estimate"). Each item's ResultCard sits collapsed below.
7. Demo honesty: if the team uses a sample receipt, the UI must say "Sample receipt" (add a `?sample=1` banner).

Demo data (`data/demo.json`), owned by Person 4

json


```
[
  {
    "pair_id": "razor-1",
    "category": "disposable razors",
    "products": [
      { "name": "", "brand": "", "retailer": "Boots", "price_gbp": null, "quant
      { "name": "", "brand": "", "retailer": "Boots", "price_gbp": null, "quant
    ],
    "equivalence": 0.9,
    "equivalence_reasons": [],
    "explanation": ""
  }
]
```

- Claude Code: **create this file with empty placeholders only. Do not fill in prices or product names.** Person 4 fills them from real retailer pages.
- Target: about 8 pairs (razors, deodorant, shaving gel, body wash/shampoo), including at least 1 strong Pink Tax case for the pitch hook and 1 reverse gap.
- The app must treat `price_gbp: null` as "not found", never crash.

Extras (only after the core works, in this order)

1. **Call it out** (about 15 min): on a High result, a button that drafts a polite, factual message to the retailer or a social post ("Your women's X costs £a per blade vs £b for the men's Y..."), with copy-to-clipboard. Turns the finding into action, which counts towards real-world impact.
2. **Pink Tax Wrapped** (about 20 min): after a receipt check, a shareable card ("This trip: £X Pink Tax, about £Y a year") to screenshot. Good for originality and UI.
3. **Fair verdict highlight**: a distinct, positive design for reverse gaps, showing the tool is honest.

Do not build: accounts, a database, a browser extension, price alerts, maps, chat.

UI/UX guidelines

- Mobile-first (the pitch demo runs on a phone or a narrow browser window). Large tap targets; the bottom tab bar is always visible.
- Palette: soft pink accent, plenty of white, dark grey text. Score colours: green (no gap or reverse), amber (moderate), pink-red (high).
- Lead with the unit price comparison: big numbers, short words.
- Loading states with plain text: "Reading your receipt...", "Finding cheaper equivalents...". Web search can take 10-20 s.
- Every error state offers a next step ("Try typing the product name instead").

Commands

```
npm install
npm run dev          # http://localhost:3000 (camera works on localhost)
npm run build        # must pass before every merge
vercel --prod        # deploy; set ANTHROPIC_API_KEY in Vercel env vars
```

Git workflow

- Branches: `feat/search-engine`, `feat/scan`, `feat/receipt`. Person 1 merges to `main`.
- Merge points: after the shell and contract are on `main` (early), at the mid checkpoint, and at feature freeze.
- Keep each feature inside its own folder to avoid conflicts. Only Person 1 touches `layout.tsx`, `lib/types.ts`, `lib/score.ts` and `components/ResultCard.tsx`.

Implementation notes for Claude Code

- Put a `extractJson(text)` helper in `lib/claude.ts`: take the last text block of the response, strip code fences, find the first `{` and last `}`, `JSON.parse`, then validate with zod. On failure, retry once with "Return valid JSON only."

- With web search, the response contains several content blocks (server tool use + results + text). Read the final `text` block(s) for the JSON.
- Images go to Claude as `{ type: "image", source: { type: "base64", media_type: "image/jpeg", data } }`. Supported formats: JPEG, PNG, GIF, WebP. iPhone HEIC photos must be converted. Drawing to a canvas and exporting as JPEG handles this.
- Set `export const maxDuration = 60` on API routes that use web search (Vercel function timeout).
- Keep prompts in `lib/prompts.ts` so they're easy to tweak during the demo prep.

Definition of done (pitch-ready)

- [ ] Deployed URL works on a phone over HTTPS
- [ ] Search: the hook product returns instantly from demo.json with a High score and a source link
- [ ] Search: a reverse-gap product shows the "Good news" state
- [ ] Scan: Capture on a real product returns a result card (fallback: file input)
- [ ] Receipt: the demo receipt produces items, Pink Tax paid and could-have-saved totals
- [ ] No price on screen without a source label
- [ ] `npm run build` passes; a 60-second backup screen recording exists